

Formal Verification Meets LLMs

Proving Code Correct, Not Just Tested

Mistral's Leanstral 1.5 turns Lean 4 theorem proving into a practical bug-finding tool -- and forces a harder question about what "AI wrote this code" should mean once proof, not just testing, is on the table

Day 106 ended on a warning: a benchmark score tells you what a model can do under ideal conditions, not what it will do at 2am inside your agent loop with nobody watching. Today's story is the first real answer to that problem, specifically for code. On July 2, Mistral released Leanstral 1.5, an open-weight, Apache 2.0-licensed model built for Lean 4 -- the proof assistant mathematicians and safety-critical engineers use to establish that a piece of logic is correct, not just that it passed the tests someone thought to write. The model saturates the miniF2F formal-math benchmark, solves 587 of 672 Putnam Competition problems, and -- more interesting for anyone shipping software rather than proving theorems -- found five previously unreported bugs while scanning 57 ordinary open-source repositories. 106 days into this series we've tracked the frontier compress toward a near-tie (Day 80) and watched trust split into separate axes of facts and neutrality (Day 106); today's axis is narrower and more concrete -- can you mathematically prove the code does what you specified, and what does that actually buy you.

100% Score on the miniF2F formal-math benchmark -- saturated	587/672 PutnamBench competition-level problems solved	\$4 Cost per PutnamBench proof, vs \$300+ for Seed-Prover 1.5	5 Previously unknown bugs found across 57 open-source repos
--	---	---	---

1) What "formal verification" actually means -- and why tests can't do it

A unit test checks that your code behaves correctly on the specific inputs you thought to try. Run 10,000 tests and you've confirmed 10,000 cases -- out of a space that's usually infinite. Formal verification is a different kind of claim entirely: instead of trying inputs, you write a mathematical property ("this function never divides by zero," "this counter never overflows its integer type") and a tool like Lean 4 either constructs an airtight proof that the property holds for every possible input, or it fails and tells you exactly where the logic breaks. It's the difference between "we tried a lot of cases and nothing broke" and "it is mathematically impossible for this specific thing to break." The catch, historically, is that writing those proofs took PhD-level expertise and days of human effort per property -- which is exactly the bottleneck Leanstral 1.5 is aimed at removing.

2) Leanstral 1.5: the numbers that make this practical, not just academic

Leanstral 1.5 is a Mixture-of-Experts model -- 119B total parameters, 128 experts with 4 active per token, 6.5B activated per inference -- trained in three stages: mid-training on proof corpora, supervised fine-tuning, then reinforcement learning through a method Mistral calls CISPO across two environments, a multiturn theorem-proving loop with live compiler feedback and a code-agent environment that simulates a real developer's filesystem and language-server access. The headline number isn't the benchmark score -- it's the price. Producing a competition-grade proof used to be the kind of task you'd hire a specialist for;

Leanstral 1.5 gets there for roughly \$4 a problem.

Leanstral 1.5	~\$4	Apache 2.0 (open weights)
Aleph Prover	~\$54–\$68	Proprietary
Seed-Prover 1.5	~\$300+	Proprietary

3) The five bugs it actually found -- and the limits of what it can prove

Mistral pointed Leanstral 1.5 at 57 ordinary open-source repositories -- not math libraries, everyday infrastructure code. The model flagged 47 violated properties; 11 of those turned out to be genuine bugs, and five had never been reported on GitHub at all. The most concrete find was a sign-function overflow in the Rust library varinteger: an edge case that crashes the program outright in debug builds and, worse, silently corrupts data in release builds -- exactly the kind of bug that testing tends to miss, because nobody wrote a test for the specific integer that triggers it. That's the real limit worth naming, though: Leanstral proves only the properties someone bothered to specify. It didn't audit "is this library correct" in some general sense -- it checked a defined list of invariants (no overflow, no out-of-bounds access, and similar) against the code, and found real violations of those specific claims. A library can pass every property Leanstral checks and still have a business-logic bug nobody thought to formalize.

THE ONE RULE

Testing tells you the code didn't fail on the inputs you tried; a Lean 4 proof tells you it can't fail on any input matching the property you wrote. Leanstral 1.5's real contribution is making that second thing affordable -- \$4 instead of hundreds of dollars per proof -- but it only covers what you specified. The riskiest bug is still the one nobody thought to formalize.

A narrow, well-specified property (memory safety, overflow, protocol invariants)	Formal verification (Leanstral / Lean 4)	Writing the property is itself expert work -- a wrong spec still "proves" nothing useful
Broad, fuzzy behavior (UX flows, business logic, subjective correctness)	Traditional testing + human review	Don't expect proof-shaped certainty on requirements that were never formalized
A security-critical library you depend on but didn't write	Run both: tests for behavior, proof for invariants	Five bugs in 57 repos means untouched dependencies likely hide more

VIRAL / OPEN-SOURCE SPOTLIGHT

Glorb: the anonymous AI rapper nobody can identify

Glorb is a pseudonymous AI-generated rap act: an exaggerated 3D cartoon character rapping in a voice modeled on SpongeBob SquarePants, over absurdist, meme-dense lyrics engineered squarely for Gen Z's sense of humor. A single track went from nothing to more than 60 million TikTok views in two weeks and triggered over 500,000 user remixes; the account has since built 811,000 TikTok followers and nearly a million monthly Spotify listeners, with its most popular song topping 11 million streams. Nobody official has confirmed who is actually behind Glorb -- fans have pieced together clues pointing to a musician from Canberra, Australia, but the creator has never come forward. That deliberate anonymity is doing real work: it turns "who made this" into its own participatory mystery, on top of a format -- a beloved children's character doing unhinged rap bars, cheap to generate and endlessly remixable -- that was already built to spread. It's the same mechanic behind every low-effort, high-output AI trend: strip out the cost and skill of production, keep the shareability, and let the audience do the rest.

60M TikTok views on the breakout track in two weeks	500K+ User-generated remixes triggered by the track	811K TikTok followers, ~1M monthly Spotify listeners
---	---	--

MARKET SIGNAL

Day 106 called trust a two-axis problem for language models -- do you trust its facts, and do you trust its neutrality. Leanstral 1.5 opens a third axis, specific to code: do you trust the logic because someone proved it, or only because it happened to pass the tests you thought to write. At \$300-plus a proof, formal verification stayed an academic tool for aerospace and cryptography teams who could justify the cost. At roughly \$4 a proof, it crosses into territory a normal engineering team can point at everyday infrastructure -- and Leanstral already found bugs nobody had reported in libraries people assumed were fine, precisely because they'd never crashed in testing. The economics of proof just crossed the line where "we tested it" stops being sufficient for code that actually matters.

PRACTICAL TAKEAWAYS

1. Reach for formal verification only where the property is narrow and nameable

Memory safety, integer overflow, and protocol invariants are provable claims; “the checkout flow behaves correctly” is not something you can hand to Lean 4. Match the tool to specifications precise enough to formalize, not to fuzzy product requirements.

2. A cheap proof is still only as good as the property you wrote

Leanstrat's ~\$4 PutnamBench cost is a floor for math problems with existing formal statements; writing the specification for your own code is the expensive part that benchmark numbers don't show -- budget time for it, not just compute.

3. Treat “found 5 bugs in 57 repos” as a base rate, not a one-off

Widely-used, well-tested open-source libraries still hid an unreported overflow bug. If formal verification is this cheap now, assume your own dependencies have similar gaps nobody has looked for yet -- and that finding them is now within reach.

TOMORROW

Tomorrow -- Day 108: China's new AI-companion rules take effect July 15, and Doubao and Qwen have already switched off features rather than comply -- what the first real regulatory deadline for emotionally sticky AI products tells us about where every other market is headed next.